

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KỲ VISOLEXNORM - CHUẨN HÓA TIẾNG VIỆT

Giảng viên hướng dẫn: Thầy Ngô Minh Nhựt
Thầy Lê Long Quốc

Sinh viên thực hiện: 23127007 Trần Công Minh
23127024 Đinh Bảo Bảo

Mục lục

1	Giới thiệu	1
2	Cơ sở lý thuyết và công trình liên quan	2
2.1	Chuẩn hóa từ vựng tiếng Việt	2
2.2	Mô hình chuỗi-sang-chuỗi và BARTpho	2
2.3	Weak supervision và gán nhãn có hỗ trợ bởi LLM	3
3	Phương pháp và dữ liệu	4
3.1	Bài toán và quy trình	4
3.2	Dữ liệu	4
3.3	Xây dựng nhãn yếu	5
4	Mô hình và huấn luyện	6
4.1	Models A/B/C	6
4.2	Thiết lập huấn luyện chung	6
4.3	Thí nghiệm có kiểm soát	7
5	Đánh giá, kết quả và thảo luận	8
5.1	Chỉ số và quy tắc lựa chọn	8
5.1.1	F1 dựa trên chỉnh sửa từ vựng	8
5.1.2	Error Reduction Rate (ERR)	9
5.1.3	Exact sentence match	9
5.1.4	Chọn checkpoint và mô hình	10
5.2	Kết quả trên Dev	10
5.3	Kết quả trên Test	11

5.3.1	So sánh A/B/C	11
5.4	Thí nghiệm nhân tố có kiểm soát	12
5.5	Phân tích lỗi	13
5.5.1	Phân bố các dạng lỗi	14
5.5.2	Các ví dụ định tính tiêu biểu	14
5.6	Tổng hợp theo các câu hỏi nghiên cứu	14
6	Ứng dụng web	17
6.1	Kiến trúc ứng dụng	17
6.2	Backend FastAPI và quy trình suy luận	18
6.3	Giao diện React	19
6.4	Vận hành cục bộ	19
7	Hạn chế	22
8	Kết luận	23

Tóm tắt nội dung

Chuẩn hóa từ vựng tiếng Việt trên mạng xã hội đưa câu viết phi chuẩn về dạng thông dụng mà vẫn giữ ý nghĩa. Đề tài dùng BARTpho-syllable theo hướng sinh chuỗi-sang-chuỗi. Model A, huấn luyện trên các cặp gold của ViLexNorm, tạo câu ứng viên từ ViSoLex. Gemini rà soát các câu này, sau đó kiểm tra chất lượng theo quy tắc cố định lọc dữ liệu trước khi dùng làm nhãn yếu. Ba cấu hình A, B và C được đánh giá trên cùng tập Dev. Model C đạt ERR 0.670380, F1 0.759233 và Exact Match 0.561905, nên được chọn cho ứng dụng. Thí nghiệm nhân tố cho thấy tập nhãn yếu lớn chỉ có lợi hơn khi số epoch phù hợp. Ứng dụng gồm React và FastAPI, chạy mô hình BARTpho tại máy chủ. Vì Test đã được dùng ở giai đoạn trước, kết quả A/B/C trên Test chỉ dùng để mô tả.

Chương 1

Giới thiệu

Văn bản tiếng Việt trên mạng xã hội thường có viết tắt, thiếu dấu và cách viết theo âm. Ví dụ, *hñay t ãi hc* có thể được chuẩn hóa thành *hôm nay tôi đi học*. Mục tiêu của bài toán là đưa văn bản về dạng thông dụng mà không biến nó thành một câu diễn đạt mới.

ViLexNorm cung cấp các cặp câu gold; ViSoLex cung cấp thêm văn bản chưa gán nhãn [4, 5]. Đề tài dùng Model A tạo câu ứng viên, dùng Gemini rà soát, rồi lọc bằng kiểm tra chất lượng để tạo nhãn yếu đã được LLM rà soát. Những nhãn này không phải dữ liệu gold do con người xác minh.

Model A chỉ dùng dữ liệu gold. Model B và C đều khởi tạo từ A và dùng thêm nhãn yếu; C dùng tập nhãn lớn hơn. Ba mô hình được so sánh trên tập Dev chung, và Model C được chọn theo ERR.

Nghiên cứu trả lời ba câu hỏi: nhãn yếu đã rà soát có giúp hơn Model A không; Model B và C khác nhau thế nào trên Dev; và kích thước tập nhãn cùng số epoch ảnh hưởng ra sao trong thí nghiệm có kiểm soát.

Đề tài xây dựng baseline BARTpho-syllable, quy trình tạo nhãn yếu, so sánh A/B/C, thí nghiệm nhân tố và ứng dụng React-FastAPI. Các kết quả chỉ được diễn giải trong các cấu hình đã thử.

Chương 2

Cơ sở lý thuyết và công trình liên quan

2.1 Chuẩn hóa từ vựng tiếng Việt

Chuẩn hóa từ vựng đưa cách viết không trang trọng về dạng chuẩn và giữ nội dung của câu. Một từ viết tắt có thể mở rộng thành nhiều từ, hoặc nhiều từ nguồn có thể gộp lại. Vì vậy, mô hình sinh chuỗi phù hợp hơn cách thay thế từng từ độc lập.

ViLexNorm cung cấp dữ liệu gold và thiết lập đánh giá cho bài toán này [4]. ViSoLex bổ sung văn bản chưa gán nhãn [5]. Trong đề tài, nhãn yếu chỉ là dữ liệu huấn luyện bổ sung, không thay thế dữ liệu gold.

2.2 Mô hình chuỗi–sang–chuỗi và BARTpho

Transformer do Vaswani và cộng sự giới thiệu năm 2017 đặt nền tảng cho các mô hình chuỗi–sang–chuỗi hiện đại bằng cơ chế self-attention thay cho tính toán tuần tự hồi quy [1]. Kiến trúc encoder–decoder phù hợp với các bài toán biến đổi câu nguồn thành câu đích. Trên kiến trúc này, BART kết hợp encoder hai chiều với decoder tự hồi quy và được tiền huấn luyện bằng nhiệm vụ khôi phục văn bản bị làm nhiễu [2]. Cách học này phù hợp với các tác vụ sinh và chuyển đổi văn bản.

BARTpho do Nguyen Luong Tran, Duong Minh Le và Dat Quoc Nguyen phát triển cho tiếng Việt; công trình xuất hiện năm 2021 và được công bố tại INTERSPEECH 2022 [3]. Mô hình có hai biến thể: BARTpho-word cho đầu vào đã phân đoạn từ và BARTpho-syllable cho đầu vào ở mức âm tiết. BARTpho là mô hình chuỗi–sang–chuỗi tiền huấn luyện đơn ngữ dành cho tiếng Việt, thay vì phụ thuộc hoàn toàn vào mô hình đa ngôn ngữ.

BARTpho kế thừa kiến trúc encoder–decoder của BART. Token đầu vào cùng thông tin vị trí đi qua encoder để tạo biểu diễn theo ngữ cảnh của toàn bộ câu nguồn. Decoder sau đó sinh từng token của câu đích, dùng self-attention trên phần đã sinh và cross-attention tới biểu diễn từ encoder.

Checkpoint BARTpho-syllable được dùng trong đề tài có 12 lớp encoder, 12 lớp decoder, kích thước ẩn 1024, 16 attention heads, tối đa 1024 vị trí và vocabulary 40,030 token.

Trong tiền huấn luyện theo hướng denoising, văn bản tiếng Việt được làm nhiễu ở đầu vào và mô hình học khôi phục câu gốc. Nhờ đó, encoder học khai thác ngữ cảnh câu nguồn, còn decoder học sinh câu đích mạch lạc. Khi fine-tune, các khả năng này được điều chỉnh cho nhiệm vụ chuẩn hóa từ vựng.

Đề tài dùng `vinai/bartpho-syllable` thay vì BARTpho-word. Văn bản mạng xã hội có nhiều viết tắt, cách viết phi chuẩn và khoảng trắng không ổn định; yêu cầu phân đoạn từ chính xác trước khi chuẩn hóa sẽ tạo thêm một bước phụ thuộc. Biến thể mức âm tiết cho phép mô hình nhận dạng văn bản gần với đầu vào ban đầu hơn. Đây là lựa chọn phù hợp với thiết kế của đề tài, không phải kết luận rằng BARTpho-syllable luôn tốt hơn BARTpho-word. Ví dụ, encoder nhận câu *h nay t đi hc* và decoder có thể sinh *hôm nay tôi đi học*; phép biến đổi này có thể thay đổi số token và khoảng trắng.

2.3 Weak supervision và gán nhãn có hỗ trợ bởi LLM

Weak supervision mở rộng dữ liệu huấn luyện khi dữ liệu gán nhãn thủ công còn ít, nhưng chất lượng dữ liệu cần được kiểm soát [6]. Mô hình ngôn ngữ lớn cũng có thể hỗ trợ gán nhãn nếu quyết định tự động được kiểm tra theo yêu cầu của tác vụ [7].

Trong đề tài, Gemini chỉ dùng để rà soát các câu ứng viên. Các câu được chấp nhận sau bước kiểm tra chất lượng được gọi là nhãn yếu đã được LLM rà soát, không phải dữ liệu gold.

Chương 3

Phương pháp và dữ liệu

3.1 Bài toán và quy trình

Cho một câu đầu vào nhiều x từ văn bản mạng xã hội, mục tiêu là sinh câu chuẩn hóa y sao cho nội dung và ý định giao tiếp của x được bảo toàn. Bài toán được mô hình hóa dưới dạng sinh có điều kiện $P(y | x; \theta)$, trong đó θ là các tham số của mô hình. Với chuỗi đích $y = (y_1, \dots, y_T)$, xác suất được phân rã tự hồi quy như sau:

$$P(y | x; \theta) = \prod_{t=1}^T P(y_t | y_{<t}, x; \theta).$$

Việc sinh chuỗi cho phép độ dài đầu ra khác độ dài đầu vào, từ đó biểu diễn được các ánh xạ một-nhiều và nhiều-một thường gặp khi mở rộng từ viết tắt, khôi phục khoảng trắng hoặc chuẩn hóa một cụm từ. Cách đặt vấn đề này không yêu cầu một bộ phân loại độc lập ở từng token.

Quy trình gồm: ViLexNorm huấn luyện Model A; Model A tạo câu ứng viên từ ViSoLex; Gemini rà soát; kiểm tra chất lượng lọc dữ liệu; Model B và C dùng các tập nhãn yếu khác nhau. Ba mô hình được đánh giá trên cùng tập Dev.

3.2 Dữ liệu

ViLexNorm được giữ theo ba tập Train, Dev và Test. Mỗi bản ghi xử lý có mã định danh, tên tập dữ liệu, split, câu đầu vào nhiều, câu đích chuẩn hóa và nguồn nhãn. Tiền xử lý chuẩn hóa khoảng trắng và loại bỏ bản ghi có đầu vào hoặc câu đích rỗng; tỉn hiệu nhiều từ vệt trong câu nguồn không bị sửa trước, vì đó là đối tượng mà mô hình cần chuẩn hóa. Train được dùng để fine-tune có giám sát bằng dữ liệu gold. Dev được dùng để giữ checkpoint trong từng cấu hình và để so sánh trong quá trình phát triển. Test không tham gia checkpoint retention hay application-model selection; lịch sử và phạm vi diễn giải các lần đánh giá Test được trình bày ở phần đánh giá.

Bảng 3.1: Các split ViLexNorm được sử dụng trong dự án.

Split	Số câu	Vai trò
Train	8,372	Fine-tune có giám sát bằng dữ liệu gold
Dev	1,050	Giữ checkpoint và so sánh phát triển
Test	1,045	Đánh giá

Sau tiền xử lý và loại trùng, ViSoLex có 68,411 câu để tạo ứng viên. Các câu trùng chính xác với Dev hoặc Test bị loại bằng so khớp chuỗi đã chuẩn hóa; cách này không phát hiện các câu gần trùng về ngữ nghĩa.

3.3 Xây dựng nhãn yếu

Model A sinh một câu ứng viên cho mỗi câu ViSoLex bằng beam search 4 beams, với độ dài tối đa 128 token. Câu ứng viên chỉ là đầu vào cho bước rà soát, chưa phải nhãn huấn luyện. Gemini quyết định KEEP để giữ câu ứng viên, EDIT để sửa câu đích, hoặc REJECT để loại bản ghi.

Sau rà soát, kiểm tra chất lượng loại câu đích rỗng, dữ liệu trùng, chồng lấn với tập được bảo vệ và các thay đổi vượt phạm vi chuẩn hóa. Tỷ số độ dài đích/nguồn phải nằm trong $[0.5, 2.0]$, tỷ lệ sửa ký tự không vượt 0.8, và không được làm mất emoji, ký hiệu, chuỗi lặp hay số của câu nguồn. Các kiểm tra này giảm lỗi rõ ràng nhưng không biến nhãn LLM thành dữ liệu gold.

Bảng 3.2: Kết quả review và số nhãn yếu được chấp nhận. Số chấp nhận có thể thấp hơn tổng KEEP và EDIT vì kiểm tra xác định diễn ra sau quyết định review.

Chỉ tiêu	Lượt ban đầu	Lượt mở rộng
KEEP	9,043	21,973
EDIT	9,952	23,931
REJECT	1,002	2,502
Nhãn yếu được chấp nhận	18,970	45,843

Hợp của hai lượt chứa 64,813 nhãn yếu được LLM rà soát. Tập 18,970 nhãn ban đầu là đầu vào của Model B; Model C dùng toàn bộ tập hợp mở rộng. Cách tổ chức này giữ rõ sự khác nhau giữa candidate thô, response review và bản ghi nhãn yếu đã vượt qua QC.

Chương 4

Mô hình và huấn luyện

4.1 Models A/B/C

Model A khởi tạo từ `vinai/bartpho-syllable` công khai và chỉ dùng dữ liệu gold. Model B khởi tạo từ A, còn Model C cũng khởi tạo trực tiếp từ A, không qua B. B và C dùng nhân yếu đã được LLM rà soát, với tỷ lệ gold:nhân yếu 1 : 1 trong mỗi epoch. Model B dùng tập nhân nhỏ hơn trong 3 epoch; Model C dùng tập lớn hơn trong 8 epoch. Vì hai yếu tố này cùng thay đổi, cần dùng thí nghiệm có kiểm soát để phân biệt ảnh hưởng của chúng.

Bảng 4.1: Cấu hình huấn luyện của ba mô hình; bảng không bao gồm chỉ số kết quả.

Thuộc tính	Model A	Model B	Model C
Checkpoint khởi tạo	BARTpho công khai	Model A	Model A
Gold mỗi epoch	8,372	8,372	8,372
Tập nhân yếu	–	18,970	64,813
Nhân yếu mỗi epoch	–	8,372	8,372
Số epoch	Tối đa 5	3	8
Learning rate	3×10^{-5}	2×10^{-5}	2×10^{-5}
Batch size / tích lũy gradient	8/2	8/2	8/2
Tiêu chí chọn checkpoint	Dev loss thấp nhất	Dev loss thấp nhất	Dev loss thấp nhất

4.2 Thiết lập huấn luyện chung

Các mô hình dùng câu nguồn và câu đích tối đa 128 token, batch size 8 và tích lũy gradient 2 bước. Weight decay là 0.01 và warmup ratio là 0.1. Trong mỗi lần huấn luyện, checkpoint có Dev loss thấp nhất được giữ lại.

4.3 Thí nghiệm có kiểm soát

Thí nghiệm dùng hai kích thước tập nhãn yếu và hai số epoch, tạo thành S3, S8, L3 và L8. Mỗi cấu hình được huấn luyện với ba seed ghép cặp 2026, 2126 và 2226. Các mô hình đều khởi tạo từ Model A và giữ tỷ lệ gold:nhãn yếu 1 : 1. Tập Test không được dùng trong thí nghiệm này.

Chương 5

Đánh giá, kết quả và thảo luận

5.1 Chỉ số và quy tắc lựa chọn

Để đảm bảo tính nhất quán và công bằng khi so sánh các mô hình, toàn bộ các lượt đánh giá trong đề tài đều tuân thủ một giao thức sinh và tính điểm xác định dùng chung. Cả ba mô hình Model A, Model B và Model C đều sử dụng cùng một tokenizer, cùng kiến trúc nền tảng và cùng một cấu hình sinh văn bản tự động. Cụ thể, quá trình sinh văn bản sử dụng phương pháp beam search với 4 beams, bật chế độ dừng sớm (*early stopping*), kích thước batch đánh giá là 16, giới hạn độ dài chuỗi nguồn là 128 token và độ dài tối đa cho chuỗi sinh mới là 128 token. Hạt giống ngẫu nhiên được đóng băng cố định tại 2026.

Mỗi lượt suy luận đánh giá nhận đầu vào là các câu từ split tương ứng (Dev hoặc Test), sinh ra các chuỗi chuẩn hóa dự đoán và lưu trữ theo định dạng có cấu trúc thống nhất. Các tệp dự đoán này sau đó được chuyển trực tiếp đến bộ chấm điểm (*scorer*) xác định duy nhất của dự án mà không trải qua bất kỳ bước hậu xử lý thủ công nào. Giao thức dùng chung giúp hạn chế sai khác do cấu hình sinh hoặc cách tính điểm, từ đó làm cho việc so sánh giữa các cấu hình nhất quán hơn.

5.1.1 F1 dựa trên chỉnh sửa từ vựng

Đánh giá chuẩn hóa từ vựng ở mức độ thay thế từ vựng (*lexical edits*) phản ánh trực tiếp khả năng phát hiện và chỉnh sửa các biến thể phi chuẩn của mô hình. Thay vì so khớp chuỗi ký tự đơn thuần, quy trình đánh giá tách chuỗi nguồn x , chuỗi đích chuẩn hóa y^* và chuỗi dự đoán $\hat{y}$ thành các token phân cách bởi khoảng trắng.

Sử dụng thuật toán so khớp chuỗi (**SequenceMatcher**), quy trình xác định các đoạn biến đổi (*non-equal spans*) giữa câu nguồn và câu đích để trích xuất tập các chỉnh sửa từ vựng. Quy trình này bảo toàn tự nhiên các phép biến đổi một–nhiều ($1 : n$) và nhiều–một ($n : 1$), phản ánh các hiện tượng như mở rộng từ viết tắt hoặc ghép cụm từ. Gọi G là tổng số chỉnh sửa từ vựng của dữ liệu chuẩn (gold edits), S là tổng số chỉnh sửa do mô hình đề xuất (predicted edits), và C là số chỉnh

sửa của mô hình trùng khớp hoàn toàn với dữ liệu chuẩn về cả vị trí nguồn lẫn nội dung thay thế (correct edits). Khi đó, các chỉ số Precision (P), Recall (R) và F_1 được định nghĩa như sau:

$$P = \frac{C}{S}, \quad R = \frac{C}{G}, \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}. \quad (5.1)$$

Nếu mô hình không đưa ra bất kỳ chỉnh sửa nào ($S = 0$), Precision được quy ước bằng 0; tương tự, nếu tập dữ liệu không có chỉnh sửa ($G = 0$), Recall được quy ước bằng 0.

5.1.2 Error Reduction Rate (ERR)

Chỉ số giảm tỷ lệ lỗi (Error Reduction Rate - ERR) là thước đo chủ đạo được đề tài sử dụng để lựa chọn mô hình triển khai ứng dụng. ERR là một độ đo được sử dụng trong đánh giá lexical normalization và trong ViLexNorm [4]. Trong dự án này, ERR được tính bằng bộ scorer xác định của dự án theo sai số token so với mô hình cơ sở giữ nguyên đầu vào (*Leave-As-Is* - LAI).

Đơn vị lỗi được xác định dựa trên khoảng cách Levenshtein ở mức token phân tách bằng khoảng trắng. Gọi E_{LAI} là tổng khoảng cách Levenshtein giữa toàn bộ các câu nguồn x và câu đích chuẩn y^* trên tập đánh giá, và E_{system} là tổng khoảng cách Levenshtein giữa chuỗi dự đoán của mô hình $\hat{y}$ và câu đích y^* . Chỉ số ERR được tính theo công thức:

$$\text{ERR} = \frac{E_{\text{LAI}} - E_{\text{system}}}{E_{\text{LAI}}}. \quad (5.2)$$

Ý nghĩa định lượng của ERR được phân định rõ ràng:

- $\text{ERR} = 1.0$: Mô hình loại bỏ hoàn toàn mọi lỗi từ vựng trong văn bản, tái tạo chính xác toàn bộ câu đích.
- $\text{ERR} = 0.0$: Mô hình không mang lại cải thiện nào so với việc giữ nguyên văn bản thô ban đầu ($E_{\text{system}} = E_{\text{LAI}}$).
- $\text{ERR} < 0.0$: Mô hình sinh ra nhiều lỗi biên tập sai hơn cả số lỗi vốn có trong câu đầu vào, làm suy giảm chất lượng văn bản so với trạng thái chưa chuẩn hóa.

5.1.3 Exact sentence match

Bên cạnh các chỉ số ở cấp độ token và chỉnh sửa cục bộ, chỉ số khớp câu chính xác (*exact sentence match*) đánh giá khả năng hoàn thiện câu ở cấp độ toàn cục. Chỉ số này biểu thị tỷ lệ phần trăm các câu trong tập đánh giá mà ở đó chuỗi dự đoán của mô hình sau khi chuẩn hóa khoảng trắng trùng khớp tuyệt đối 100% với chuỗi câu đích chuẩn:

$$\text{ExactMatch} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i = y_i^*), \quad (5.3)$$

trong đó N là tổng số câu trong tập đánh giá và $\mathbf{1}(\cdot)$ là hàm chỉ thị nhận giá trị 1 khi hai chuỗi ký tự đồng nhất và 0 trong trường hợp ngược lại.

5.1.4 Chọn checkpoint và mô hình

Trong toàn bộ nghiên cứu, có hai cấp độ lựa chọn cần được phân biệt một cách tường minh và nghiêm ngặt:

1. **Giữ checkpoint nội bộ từng lượt chạy (*within-run checkpoint retention*):** Trong suốt quá trình huấn luyện của một cấu hình cụ thể (Model A, B, hay C), checkpoint được lưu lại để đại diện cho cấu hình đó là checkpoint đạt giá trị mất mát kiểm định (Dev loss) nhỏ nhất.
2. **Lựa chọn mô hình ứng dụng liên mô hình (*across-model application selection*):** Sau khi đã xác định được checkpoint đại diện của từng cấu hình, các checkpoint này được đưa vào đánh giá đồng thời trên cùng một tập Dev gồm 1,050 mẫu chung. Việc chọn mô hình nào làm mô hình mặc định phục vụ ứng dụng người dùng cuối tuân thủ thứ tự ưu tiên đóng băng: (1) ERR cao hơn; (2) F1 cao hơn; (3) Exact match cao hơn; (4) Quy tắc ưu tiên Model A khi hòa điểm.

Model C đạt thứ hạng cao nhất theo quy tắc trên và được chọn làm mô hình mặc định cho dịch vụ suy luận của ứng dụng, trong khi Model B được chỉ định làm phương án dự phòng (*fallback*). Toàn bộ quá trình chọn mô hình ứng dụng được thực hiện độc quyền trên tập Dev. Các chỉ số trên tập kiểm tra (Test) tuyệt đối không tham gia vào quyết định lựa chọn này.

5.2 Kết quả trên Dev

Bảng 5.1 trình bày kết quả đánh giá chung của ba mô hình trên tập Dev của ViLexNorm gồm 1,050 câu với 1,784 chỉnh sửa chuẩn và 2,424 lỗi token theo mô hình cơ sở Leave-As-Is.

Dựa trên quy tắc lựa chọn đã đóng băng, Model C đạt ERR cao nhất (0.670380), đồng thời dẫn đầu về cả F_1 (0.759233) và Exact Match (0.561905). Do đó, Model C được chọn làm mô hình phục vụ mặc định trong hệ thống ứng dụng, và Model B được giữ lại làm phương án dự phòng.

Về mặt định lượng, Model B đạt mức tăng +0.0569 ERR và +0.0386 F_1 so với baseline gold-only Model A. Khi mở rộng tập nhãn yếu ở Model C, hiệu năng tiếp tục tăng thêm +0.0095 điểm ERR và +0.0057 điểm F_1 . Mặc dù Model C đạt kết quả cao hơn Model B trên tập Dev, sự khác biệt giữa

Bảng 5.1: Kết quả đánh giá trên tập Dev chung (1,050 mẫu) và thứ hạng lựa chọn mô hình ứng dụng.

Mô hình	ERR	F1	Exact Match	Thứ hạng
Model A	0.603960	0.714903	0.520952	3
Model B	0.660891	0.753515	0.557143	2
Model C	0.670380	0.759233	0.561905	1

hai cấu hình này trong lịch sử huấn luyện đồng thời bao gồm cả quy mô pool nhãn yếu (18,970 so với 64,813) lẫn số lượng epoch huấn luyện (3 epoch so với 8 epoch). Do đó, chưa thể khẳng định rằng việc mở rộng quy mô dữ liệu đơn thuần là nguyên nhân trực tiếp dẫn đến sự cải thiện của Model C nếu không kiểm soát yếu tố thời lượng huấn luyện.

5.3 Kết quả trên Test

Để phản ánh trung thực tiến trình nghiên cứu, cần ghi nhận rằng lịch sử đánh giá trên tập Test không diễn ra theo kịch bản lý tưởng là giữ nguyên một tập holdout tinh khôi để kiểm thử một lần duy nhất sau khi hoàn thành mọi mô hình. Tiến trình thực tế bao gồm hai giai đoạn rõ rệt:

Ở giai đoạn phát triển ban đầu của dự án, khi chỉ mới xây dựng Model A và Model B, một lượt đánh giá Test đã được thực hiện trên 1,045 mẫu của ViLexNorm Test. Kết quả lịch sử ghi nhận tại thời điểm đó như sau:

- **Model A:** ERR đạt 0.600250; F_1 đạt 0.718184.
- **Model B:** ERR đạt 0.633111; F_1 đạt 0.742215.

Trong khuôn khổ đánh giá cục bộ tại thời điểm đó, Model B đạt các chỉ số Test cao hơn Model A và được lựa chọn ở giai đoạn ấy. Đây là bằng chứng lịch sử ghi nhận kết quả so sánh ban đầu giữa hai cấu hình, không phải là quy tắc chọn mô hình chung cuộc của toàn bộ đề tài.

5.3.1 So sánh A/B/C

Sau khi Model C được huấn luyện và được lựa chọn cho ứng dụng dựa trên kết quả tập Dev, một benchmark tổng hợp so sánh cả ba mô hình Model A, B và C đã được thực hiện lại trên tập Test 1,045 mẫu. Vì tập Test này đã từng được nạp và quan sát ở giai đoạn đánh giá Model A/B trước đó, benchmark này hoàn toàn mang tính *hậu nghiệm* (*post-hoc*) và *mô tả* (*descriptive*). Nó tuyệt đối không đóng vai trò là tập kiểm thử độc lập tinh khôi để chứng minh năng lực tổng quát hóa, cũng không tham gia vào việc lựa chọn hay thăng hạng mô hình cho ứng dụng.

Bảng 5.2: Kết quả benchmark hậu nghiệm mang tính mô tả trên tập ViLexNorm Test (1,045 mẫu).

Mô hình	ERR	Precision	F1	Exact Match
Model A	0.600250	0.733999	0.718184	0.503349
Model B	0.633111	0.761538	0.742215	0.529187
Model C	0.664725	0.780902	0.764993	0.566507

Kết quả đo đặc chi tiết trên 1,045 mẫu Test (với 1,778 chỉnh sửa gold và 2,404 lỗi token LAI) được trình bày trong Bảng 5.2.

Trong 1,778 chỉnh sửa gold trên tập Test, Model A dự đoán 1,703 chỉnh sửa và đúng 1,250; Model B dự đoán 1,690 chỉnh sửa và đúng 1,287; Model C dự đoán 1,707 chỉnh sửa và đạt 1,333 chỉnh sửa chính xác. Về mặt mô tả, Model C đạt điểm số cao nhất trên cả bốn thước đo ERR (0.664725), Precision (0.780902), F_1 (0.764993) và Exact Match (0.566507). Thứ hạng này hoàn toàn tương thích và nhất quán với kết quả quan sát được trên tập Dev.

Để lượng hóa độ không chắc chắn và mức biến thiên của chênh lệch hiệu năng theo cặp trên mẫu Test đã quan sát, phương pháp ghép cặp bootstrap với 2,000 lần lấy mẫu lại ngẫu nhiên (hạt giống 2026) được thực hiện. Kết quả cho thấy:

- So sánh giữa Model C và Model B: Chênh lệch ΔF_1 là +0.022778 với khoảng tin cậy 95% bootstrap là [0.012063, 0.033228]; chênh lệch ΔERR là +0.031614 với khoảng tin cậy 95% là [0.015893, 0.046803].
- So sánh giữa Model C và Model A: Chênh lệch ΔF_1 là +0.046808 với khoảng tin cậy 95% là [0.033537, 0.060202]; chênh lệch ΔERR là +0.064476 với khoảng tin cậy 95% là [0.045868, 0.083898].

Mặc dù khoảng tin cậy 95% bootstrap không chứa giá trị 0, cần nhấn mạnh rằng kết quả này chỉ lượng hóa độ biến thiên trên tập dữ liệu đã quan sát trước đó, chứ không biến phép thử hậu nghiệm thành bằng chứng xác nhận độc lập về năng lực tổng quát hóa.

5.4 Thí nghiệm nhân tố có kiểm soát

Như đã phân tích ở Phần 5.2, cấu hình huấn luyện của Model B và Model C có sự chồng lấn (*confounding*) giữa quy mô pool nhân yếu và thời lượng huấn luyện. Thí nghiệm nhân tố có kiểm soát (2×2 factorial) được thực hiện độc quyền trên tập Dev (không nạp tập Test) với hai quy mô pool (18,970 và 64,813) và hai số epoch (3 và 8), tạo thành bốn ô thực nghiệm S3, S8, L3 và L8 trên ba seed ghép cặp (2026, 2126, 2226).

Bảng 5.3 tổng hợp các giá trị trung bình trên ba hạt giống của các checkpoint có Dev loss thấp nhất.

Bảng 5.3: Kết quả trung bình trên ba hạt giống của thí nghiệm nhân tố có kiểm soát trên Dev.

Ô	Quy mô pool	Số epoch	ERR	F1	Exact Match
S3	18,970	3	0.652778	0.747295	0.548889
S8	18,970	8	0.662679	0.754182	0.553651
L3	64,813	3	0.649065	0.744360	0.543175
L8	64,813	8	0.670380	0.761249	0.563175

Phân tích các tác động biên và tương tác từ dữ liệu thực nghiệm cho thấy những hiện tượng đáng chú ý:

1. **Tác động của quy mô pool ở 3 epoch:** Chênh lệch giữa L3 và S3 là âm trên cả ba chỉ số (ERR giảm -0.003713 , F_1 giảm -0.002936 , Exact Match giảm -0.005714). Kết quả này cho thấy trong thiết kế hiện tại, việc tăng quy mô pool không tạo lợi thế ở 3 epoch.
2. **Tác động của quy mô pool ở 8 epoch:** Khi huấn luyện 8 epoch, cấu hình L8 đạt kết quả cao hơn S8 (ERR tăng $+0.007701$, F_1 tăng $+0.007067$, Exact Match tăng $+0.009524$).
3. **Tác động của thời lượng huấn luyện:** Kéo dài từ 3 lên 8 epoch giúp cải thiện hiệu năng ở cả hai mức pool, nhưng mức tăng ở pool mở rộng (L8 so với L3: ERR $+0.021315$, F_1 $+0.016889$) lớn hơn so với mức tăng ở pool nhỏ (S8 so với S3: ERR $+0.009901$, F_1 $+0.006887$).
4. **Tương tác giữa hai yếu tố:** Tác động tương tác giữa quy mô pool và thời lượng huấn luyện mang dấu dương (ERR $+0.011414$, F_1 $+0.010002$, Exact Match $+0.015238$).

Kết luận then chốt được rút ra là: **L8 là cấu hình mạnh nhất trong toàn bộ không gian thực nghiệm dưới quy thức chọn checkpoint theo Dev loss thấp nhất**. Tuy nhiên, phân tích độ nhạy tại checkpoint kết thúc (terminal epoch 8) cho thấy chênh lệch giữa L8 và S8 xấp xỉ 0 (ERR -0.000138 , F_1 $+0.000316$). Do đó, ưu thế của L8 gắn liền với quy thức lưu giữ checkpoint tối ưu trên Dev chứ không phải một quy luật nhân quả tuyệt đối cho rằng tăng pool luôn tự động mang lại cải thiện.

Một thí nghiệm bổ sung cho phép huấn luyện tối đa 20 epoch. Checkpoint được chọn ở epoch 9 đạt ERR 0.668729, thấp hơn L8 (0.670380), nên không làm thay đổi lựa chọn mô hình.

5.5 Phân tích lỗi

Để mô tả chi tiết hơn các dạng đầu ra và lỗi của mô hình, một phân tích định lượng và định tính được tiến hành trên 1,045 câu của tập kiểm tra.

5.5.1 Phân bố các dạng lỗi

Bảng 5.4 phân loại các câu dự đoán thành các nhóm nhận diện xác định theo cơ chế heuristic.

Bảng 5.4: Phân bố các phân loại kết quả trên 1,045 câu kiểm tra.

Phân loại	Model A		Model B		Model C	
	Số câu	Tỷ lệ (%)	Số câu	Tỷ lệ (%)	Số câu	Tỷ lệ (%)
Chính xác tuyệt đối (Correct)	526	50.3%	553	52.9%	592	56.7%
Dự đoán sai (Wrong)	350	33.5%	327	31.3%	302	28.9%
Bỏ sót chuẩn hóa (Missed)	54	5.2%	64	6.1%	59	5.6%
Chỉnh sửa một-nhiều (One-to-many)	113	10.8%	99	9.5%	90	8.6%
Chỉnh sửa nhiều-một (Many-to-one)	2	0.2%	2	0.2%	2	0.2%
Tổng	1,045	100.0%	1,045	100.0%	1,045	100.0%

Dữ liệu cho thấy số lượng câu chuẩn hóa đúng hoàn toàn tăng dần từ Model A (526 câu, 50.3%) lên Model B (553 câu, 52.9%) và đạt cao nhất ở Model C (592 câu, 56.7%). Song song đó, tỷ lệ câu dự đoán sai giảm đều đặn từ 33.5% xuống 28.9%. Cần lưu ý rằng các danh mục lỗi trên được xác định dựa trên so khớp hình thái bề mặt và cấu trúc đoạn chỉnh sửa của token, phản ánh các mẫu biến tập cấu trúc chứ không đại diện cho các nguyên nhân tâm lý ngôn ngữ học hay cơ chế suy luận bên trong mạng nơ-ron.

Khi so sánh kết quả từng cặp câu cụ thể giữa Model C và các mô hình tiền nhiệm:

- Giữa Model C và Model A: Có 91 câu Model C sửa đúng nhưng Model A sửa sai; ngược lại, có 25 câu Model A sửa đúng nhưng Model C lại sửa sai (929 câu còn lại hòa điểm).
- Giữa Model C và Model B: Có 57 câu Model C sửa đúng nhưng Model B sửa sai; ngược lại, có 18 câu Model B sửa đúng nhưng Model C sửa sai (970 câu hòa điểm).

Sự tồn tại của các trường hợp thoái lui (*regressions*) cho thấy rằng xu hướng cải thiện tổng thể vẫn đi kèm một số trường hợp thoái lui cục bộ.

5.5.2 Các ví dụ định tính tiêu biểu

Bảng 5.5 minh họa bốn trường hợp điển hình được trích xuất từ dữ liệu kiểm tra, thể hiện rõ các hành vi ngôn ngữ học của các mô hình.

5.6 Tổng hợp theo các câu hỏi nghiên cứu

Phần này tổng hợp các kết quả thực nghiệm để trả lời trực tiếp ba câu hỏi nghiên cứu đã được đặt ra ở Phần mở đầu:

Bảng 5.5: Các ví dụ định tính tiêu biểu từ tập kiểm tra.

Trường hợp	Chi tiết câu và dự đoán	Nhận xét hành vi
Lựa chọn đại từ phù hợp ngữ cảnh	Nguồn: <i>giống khẩu phần hồi covid của t ha m ha</i> Đích: giống khẩu phần hồi covid của tao ha mày ha Model A: ... của <u>tôi</u> ha mày ha Model B: ... của tao ha mày ha Model C: ... của tao ha mày ha	Model A sinh <i>tôi</i> , trong khi Model B và Model C sinh <i>tao</i> , khớp với câu đích trong trường hợp này.
Khử biến thể phức tạp	Nguồn: <i>nhều khi teo tưởng con mèo nó biết nói thật ạ</i> Đích: nhiều khi tao tưởng con mèo nó biết nói thật ạ Model A: nhiều khi <u>tôi</u> tưởng ... Model B: nhiều khi <u>tôi</u> tưởng ... Model C: nhiều khi tao tưởng ...	Model A và Model B sinh <i>tôi</i> , trong khi Model C sinh <i>tao</i> , khớp với câu đích chuẩn.
Bỏ sót chuẩn hóa do viết hoa toàn bộ	Nguồn: <i>AE mà sao kỳ vậy.</i> Đích: Anh em mà sao kỳ vậy. Model A: <u>AE</u> mà sao kỳ vậy. Model B: <u>AE</u> mà sao kỳ vậy. Model C: <u>AE</u> mà sao kỳ vậy.	Cả ba mô hình giữ nguyên <i>AE</i> trong trường hợp này, cho thấy một kiểu lỗi bỏ sót chung được quan sát ở ví dụ trên.
Trường hợp thoái lui của Model C	Nguồn: <i>Nhìn nóa lại nghĩ đến con ngỗng nhà toy</i> Đích: Nhìn nó lại nghĩ đến con ngỗng nhà tôi Model A: Nhìn nó lại nghĩ ... nhà tôi Model B: Nhìn nó lại nghĩ ... nhà tôi Model C: Nhìn <u>nóa</u> lại nghĩ ... nhà tôi	Model A và Model B chuẩn hóa cả hai từ nhiều (<i>nó, tôi</i>). Model C chuẩn hóa đúng <i>toy</i> → <i>tôi</i> nhưng giữ nguyên <i>nóa</i> trong trường hợp này.

Trả lời RQ1: Các cấu hình sử dụng nhân yếu đã được LLM rà soát thể hiện như thế nào so với *fine-tune* chỉ bằng dữ liệu *gold*?

Trên common Dev, các cấu hình sử dụng LLM-reviewed weak labels đạt các chỉ số cao hơn gold-only Model A. Cụ thể, Model B đạt ERR 0.660891 so với 0.603960 và F_1 0.753515 so với 0.714903 của Model A. Kết quả benchmark hậu nghiệm trên Test cũng ghi nhận xu hướng nhất quán về mặt mô tả. Kết quả này cho thấy weak-label pipeline cung cấp tín hiệu giám sát bổ sung có ích trong các cấu hình được đánh giá.

Trả lời RQ2: Cấu hình dùng tập nhân đã rà soát ban đầu và cấu hình dùng tập mở rộng khác nhau như thế nào khi được đánh giá trên cùng tập Dev?

Khi đánh giá trên cùng tập Dev chung, Model C (sử dụng tập nhân yếu mở rộng 64,813 mẫu) đạt kết quả cao hơn Model B (sử dụng tập nhân ban đầu 18,970 mẫu) trên tất cả các thước đo: ERR đạt 0.670380 so với 0.660891 (+0.0095); F_1 đạt 0.759233 so với 0.753515 (+0.0057); và Exact Match đạt 0.561905 so với 0.557143 (+0.0048). Nhờ đó, Model C giành vị trí số 1 và trở thành mô

hình mặc định cho ứng dụng. Tuy nhiên, mức tăng từ B lên C nhỏ hơn so với mức tăng từ A lên B. Đồng thời, do Model B và Model C khác nhau cả về dung lượng pool lẫn số epoch huấn luyện trong lịch sử, kết quả này chưa thể khẳng định rằng việc mở rộng pool nhân yếu luôn tự động tạo ra cải thiện hiệu năng.

Trả lời RQ3: *Các khác biệt quan sát được liên hệ như thế nào với độ sẵn có của tập nhân đã rà soát và thời lượng huấn luyện?*

Thí nghiệm nhân tố có kiểm soát (2×2 factorial) cho thấy một tương tác dương quan sát được giữa quy mô pool và thời lượng huấn luyện:

- Ở 3 epoch, pool mở rộng không tạo lợi thế so với pool nhỏ (L3 đạt ERR 0.649065, thấp hơn S3 với 0.652778).
- Ở 8 epoch, pool mở rộng đạt kết quả cao hơn (L8 đạt ERR 0.670380, cao hơn S8 với 0.662679).
- Tác động gia tăng thời lượng huấn luyện từ 3 lên 8 epoch đem lại mức cải thiện ở pool mở rộng (+0.021315 ERR) lớn hơn so với pool nhỏ (+0.009901 ERR), tạo nên tương tác dương (+0.011414 ERR).

Trong thiết kế factorial này, cấu hình expanded-pool/8-epoch (L8) đạt kết quả trung bình cao nhất dưới quy tắc giữ checkpoint theo minimum Dev loss. C-max20 không cung cấp bằng chứng rằng việc tiếp tục mở rộng training horizon ngoài cấu hình L8 sẽ cải thiện checkpoint được lựa chọn theo cùng nguyên tắc. Do đó, kết quả hỗ trợ vai trò kết hợp giữa pool availability và training duration trong phạm vi các cấu hình đã thử, nhưng không xác lập 8 epoch là một optimum tổng quát.

Chương 6

Ứng dụng web

6.1 Kiến trúc ứng dụng

Ứng dụng web được xây dựng nhằm minh họa khả năng triển khai thực tế của mô hình chuẩn hóa từ vựng tiếng Việt đã được huấn luyện và lựa chọn từ thực nghiệm. Hệ thống tuân thủ kiến trúc phân tách rõ ràng giữa tầng giao diện người dùng (*frontend*) và tầng dịch vụ nghiệp vụ (*backend*):

- **Giao diện người dùng (Frontend):** Được xây dựng bằng thư viện React kết hợp công cụ đóng gói Vite, ngôn ngữ TypeScript và khung giao diện Tailwind CSS. Tầng này chịu trách nhiệm thu nhận văn bản đầu vào từ người dùng, gửi yêu cầu chuẩn hóa qua giao thức HTTP REST, hiển thị kết quả trực quan và quản lý lịch sử thao tác cục bộ trên trình duyệt.
- **Dịch vụ xử lý (Backend):** Được xây dựng trên nền tảng FastAPI (Python), quản lý vòng đời ứng dụng và cung cấp các REST API. Backend tiếp nhận yêu cầu, kiểm tra tính hợp lệ của chuỗi đầu vào, và kích hoạt tầng suy luận mô hình.
- **Tầng suy luận (*Inference layer*):** Module `visolexnorm.app.inference` chịu trách nhiệm nạp tokenizer, trọng số checkpoint và thực thi quá trình sinh chuỗi tự hồi quy với PyTorch trong chế độ `torch.inference_mode()`.
- **Cơ chế lựa chọn mô hình và dự phòng (*Fallback*):** Checkpoint phục vụ suy luận được giải quyết tự động tại thời điểm khởi động thông qua tệp cấu hình ứng dụng và kết quả chọn mô hình từ tập Dev (`outputs/app/model_selection.json`). Mô hình mặc định là Model C (được lưu tại `checkpoints/model_c/`). Nhằm tăng khả năng phục hồi khi checkpoint mặc định không khả dụng hoặc không vượt qua kiểm tra toàn vẹn, hàm `resolve_checkpoint` thực hiện kiểm tra tính toàn vẹn của thư mục checkpoint thông qua mã băm SHA-256. Nếu checkpoint Model C không tồn tại hoặc xảy ra sai lệch tính toàn vẹn, hệ thống tự động kích hoạt cơ chế dự phòng (*fallback*), chuyển sang nạp checkpoint Model B đã được kiểm chứng (tại `checkpoints/model_b/`).

Toàn bộ quá trình suy luận ứng dụng được thực thi hoàn toàn cục bộ trên máy chủ thông qua mô hình BARTpho đã fine-tune. Mô hình ngôn ngữ lớn thương mại (Gemini) chỉ tham gia vào quy trình rà soát nhãn yếu ngoại tuyến trong giai đoạn chuẩn bị dữ liệu và hoàn toàn không chạy trong runtime của ứng dụng. Nhờ đó, hệ thống không đòi hỏi kết nối Internet tới các dịch vụ API bên ngoài khi vận hành chuẩn hóa cho người dùng cuối.

6.2 Backend FastAPI và quy trình suy luận

Backend FastAPI được tổ chức tinh gọn và thiết lập các cơ chế kiểm soát lỗi nghiêm ngặt:

1. **Khởi động và làm ấm mô hình (*Lifespan pre-warming*):** Khi dịch vụ FastAPI khởi chạy, sự kiện `lifespan` sẽ kích hoạt đọc cấu hình chọn mô hình, nạp trọng số và tokenizer vào bộ nhớ RAM, đồng thời thực thi một lượt chuẩn hóa giả lập (*dummy inference*) với chuỗi văn bản mẫu. Cơ chế này giúp giảm overhead khởi tạo đối với yêu cầu suy luận đầu tiên của người dùng.
2. **Endpoint chuẩn hóa chính (POST /api/normalize):** Tiếp nhận yêu cầu với phần thân (*payload*) dạng JSON tuân thủ schema Pydantic gồm trường `text` (chuỗi văn bản từ 1 đến 1,000 ký tự). Quy trình xử lý gồm:
 - Chuẩn hóa khoảng trắng đầu vào; từ chối chuỗi rỗng với mã lỗi HTTP 400.
 - Sử dụng tokenizer của BARTpho để đếm số token; nếu vượt quá ngưỡng giới hạn 128 token (`max_source_length`), hệ thống trả về lỗi HTTP 400 kèm thông báo rõ ràng.
 - Thực thi suy luận mô hình với giải thuật beam search (4 beams, độ dài tối đa 128 token) trong khối `torch.inference_mode()`.
 - Đo đặc thời gian xử lý thực tế và trả về phản hồi JSON chứa: `status` (trạng thái thành công), `original_text` (văn bản gốc), `normalized_text` (văn bản đã chuẩn hóa), và `latency_ms` (thời gian xử lý tính bằng mili-giây).
3. **Các endpoint kiểm tra hệ thống:** GET /api/health cung cấp tín hiệu kiểm tra tình trạng dịch vụ (*health check*) cho hạ tầng giám sát. GET /api/model-info trả về thông tin chi tiết về mô hình đang hoạt động, đường dẫn checkpoint và trạng thái kích hoạt fallback.
4. **Cấu hình CORS:** Middleware CORS cho phép các frontend origin được cấu hình truy cập API trong quá trình phát triển và kiểm thử.

6.3 Giao diện React

Giao diện người dùng được thiết kế hiện đại, tập trung vào trải nghiệm người dùng với nhận diện thương hiệu ViSoLexNorm. Ứng dụng bao gồm trang chủ thực hiện chuẩn hóa và trang giới thiệu đề tài (/about):

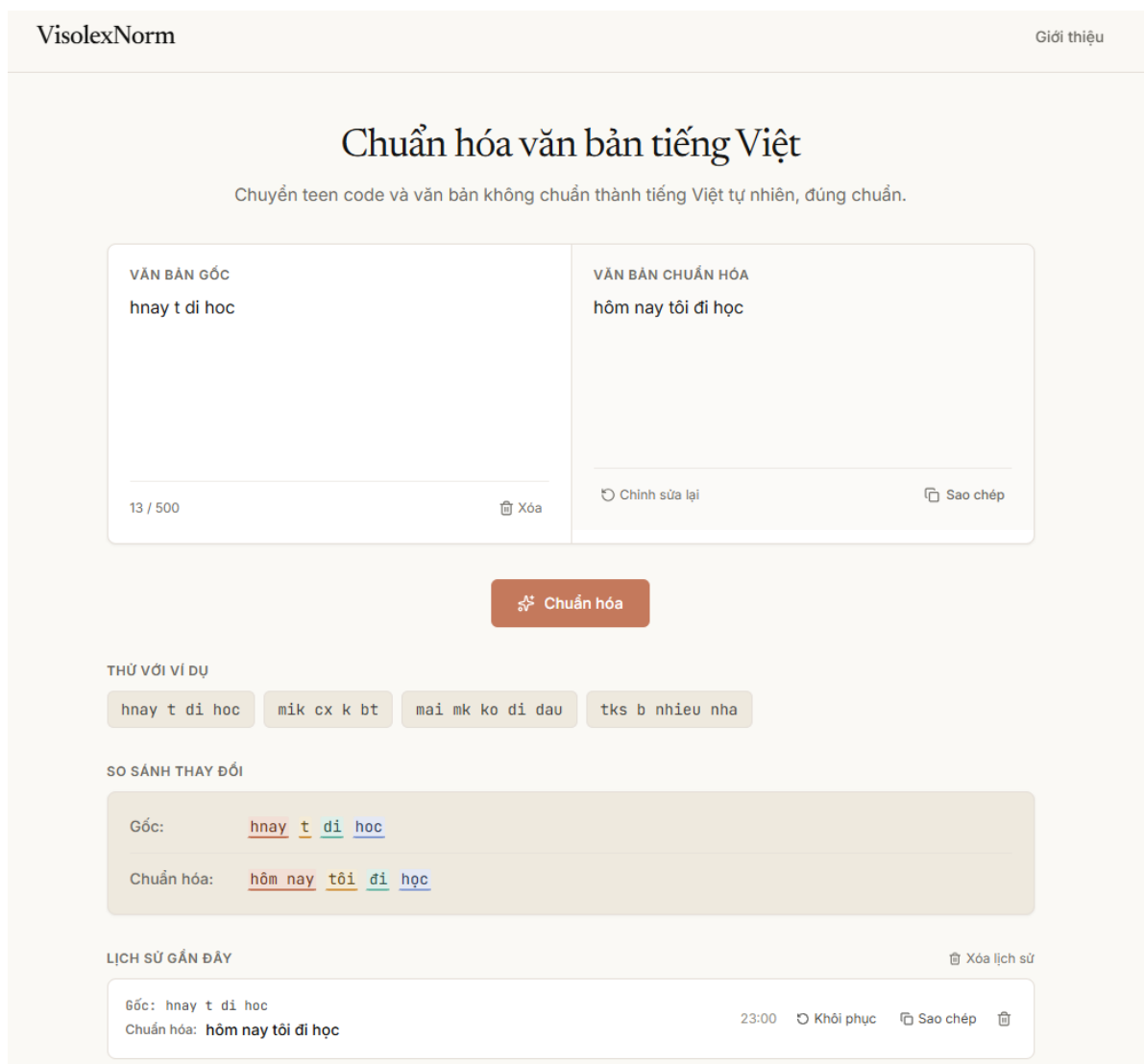
- **Không gian nhập liệu (*Normalization Workspace*):** Cung cấp khung nhập văn bản trực quan với khả năng tự động co giãn, hỗ trợ nút xóa nhanh toàn bộ nội dung và hiển thị trạng thái đang xử lý.
- **Văn bản mẫu gợi ý (*Example Inputs*):** Cung cấp sẵn các mẫu câu teencode và viết tắt thường gặp trên mạng xã hội Việt Nam, cho phép người dùng nạp nhanh vào ô nhập liệu để thử nghiệm tính năng chỉ với một thao tác nhấp chuột.
- **Nút kích hoạt chuẩn hóa (*Normalize Button*):** Hiển thị trạng thái tải (*loading spinner*) trong suốt thời gian backend xử lý suy luận, tự động vô hiệu hóa khi ô nhập liệu trống nhằm ngăn chặn các yêu cầu không hợp lệ.
- **Hộp so sánh trực quan (*Difference Box*):** Sau khi chuẩn hóa thành công, giao diện hiển thị đồng thời văn bản kết quả và hộp đối sánh trực quan, làm nổi bật các từ vựng phi chuẩn đã được biến đổi thành dạng chuẩn, giúp người dùng dễ dàng kiểm tra các chỉnh sửa cục bộ.
- **Lịch sử chuẩn hóa (*History List*):** Lưu trữ cục bộ danh sách các câu đã chuẩn hóa gần đây trên trình duyệt. Người dùng có thể xem lại kết quả cũ, sao chép nhanh vào khay nhớ tạm (*clipboard*), nạp lại câu nguồn vào khung chỉnh sửa, hoặc xóa bỏ lịch sử khi cần thiết.
- **Xử lý phản hồi và thông báo lỗi:** Các trường hợp lỗi mạng, máy chủ gián đoạn hoặc câu nhập vượt quá giới hạn độ dài đều được hiển thị thông qua thông báo lỗi thân thiện (*ErrorMessage*) ở đầu không gian làm việc.

6.4 Vận hành cục bộ

Nhằm phục vụ mục đích kiểm tra và tái lập thực nghiệm, toàn bộ ứng dụng web có thể được khởi chạy cục bộ trên môi trường máy tính cá nhân mà không đòi hỏi tài nguyên đám mây:

1. **Cài đặt môi trường Python cho ứng dụng:** Cài đặt các gói phụ thuộc tối thiểu phục vụ ứng dụng từ tệp `requirements-app.txt`:

```
pip install -r requirements-app.txt
```



Hình 6.1: Giao diện ứng dụng web ViSoLexNorm.

2. **Khởi chạy dịch vụ backend:** Từ thư mục gốc của repository, thực thi lệnh:

```
python backend/main.py
```

Máy chủ FastAPI sẽ khởi động tại địa chỉ `http://127.0.0.1:8000`, tự động tải checkpoint đã chọn và mở tài liệu tương tác Swagger tại `/docs`.

3. **Khởi chạy giao diện frontend:** Mở cửa sổ dòng lệnh thứ hai, chuyển vào thư mục giao diện và khởi động máy chủ phát triển Vite:

```
cd frontend
pnpm install --frozen-lockfile
pnpm dev
```

Giao diện web sẽ sẵn sàng truy cập trên trình duyệt tại địa chỉ `http://localhost:5173`.
Cấu hình địa chỉ máy chủ API được quản lý qua biến môi trường `VITE_API_BASE_URL` trong tệp `frontend/.env` (mặc định trỏ về cổng 8000 của backend).

Để hệ thống vận hành trơn tru, các thư mục checkpoint tương ứng (`checkpoints/model_c/` và `checkpoints/model_b/`) cần được đặt sẵn tại thư mục gốc theo đúng cấu trúc của repository.

Chương 7

Hạn chế

1. **Chất lượng nhân yếu:** Gemini giúp rà soát dữ liệu nhưng nhân được tạo ra không phải dữ liệu gold do con người xác minh. Các quy tắc kiểm tra chất lượng loại được nhiều lỗi rõ ràng nhưng không bảo đảm mọi câu đều đúng về nghĩa. Có thể đánh giá thủ công một mẫu dữ liệu trong nghiên cứu tiếp theo.
2. **Dữ liệu và quy trình đánh giá:** Kiểm tra trùng lặp chỉ phát hiện các câu trùng chính xác, không phát hiện mọi câu gần trùng về ngữ nghĩa. Tập Dev được dùng cho nhiều bước lựa chọn. Tập Test đã được sử dụng ở giai đoạn A/B, nên kết quả A/B/C về sau chỉ dùng để mô tả.
3. **Phạm vi thí nghiệm:** Model B và C lịch sử khác cả kích thước tập nhân yếu và số epoch. Thí nghiệm có kiểm soát chỉ khảo sát hai kích thước tập, hai số epoch và ba seed. Vì vậy, kết luận chỉ áp dụng cho các cấu hình đã thử.
4. **Bộ chấm điểm và ứng dụng:** Bộ chấm điểm nội bộ được dùng thống nhất cho các mô hình nhưng có thể khác chi tiết so với bộ đánh giá của công trình khác. Ứng dụng web hiện được xây dựng để trình diễn mô hình cục bộ và chưa được đánh giá cho hệ thống có tải lớn.

Chương 8

Kết luận

Đề tài xây dựng hệ thống chuẩn hóa từ vựng tiếng Việt trên mạng xã hội bằng BARTpho-syllable. Từ dữ liệu gold ViLexNorm, Model A được huấn luyện để tạo câu ứng viên cho ViSoLex. Gemini và các quy tắc kiểm tra chất lượng sau đó được dùng để xây dựng tập nhãn yếu cho Model B và C.

Trên tập Dev, Model C đạt ERR 0.670380 và được chọn làm mô hình mặc định cho ứng dụng. Thí nghiệm có kiểm soát cho thấy tập nhãn yếu lớn không tốt hơn ở 3 epoch nhưng đạt kết quả cao hơn ở 8 epoch. Vì vậy, ảnh hưởng của kích thước tập nhãn cần được xem cùng với số epoch huấn luyện trong các cấu hình đã thử.

Ứng dụng React-FastAPI cho phép sử dụng mô hình qua giao diện web. Các hạn chế chính nằm ở chất lượng nhãn yếu, việc sử dụng lại Dev/Test và phạm vi thí nghiệm còn nhỏ. Các nghiên cứu tiếp theo có thể bổ sung đánh giá thủ công cho nhãn yếu và sử dụng một tập kiểm tra mới.

Bibliography

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30*, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [2] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020. <https://aclanthology.org/2020.acl-main.703/>
- [3] Nguyen Luong Tran, Duong Minh Le, and Dat Quoc Nguyen. BARTpho: Pre-trained Sequence-to-Sequence Models for Vietnamese. In *Proceedings of the 23rd Annual Conference of the International Speech Communication Association*, 2022. <https://arxiv.org/abs/2109.09701>
- [4] Thanh-Nhi Nguyen, Thanh-Phong Le, and Kiet Nguyen. ViLexNorm: A Lexical Normalization Corpus for Vietnamese Social Media Text. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1421–1437, 2024. <https://aclanthology.org/2024.eacl-long.85/>
- [5] Anh Thi-Hoang Nguyen, Dung Ha Nguyen, and Kiet Van Nguyen. ViSoLex: An Open-Source Repository for Vietnamese Social Media Lexical Normalization. arXiv preprint arXiv:2501.07020, 2025. <https://arxiv.org/abs/2501.07020>
- [6] Dong-Hyun Lee. Pseudo-Label: The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks. In *ICML 2013 Workshop on Challenges in Representation Learning*, 2013.
- [7] Fabrizio Gilardi, Meysam Alizadeh, and Mael Kubli. ChatGPT outperforms crowd-workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120, 2023. <https://doi.org/10.1073/pnas.2305016120>